BA ZC581 :
Applied Data Science
Module: 8

J A Gokhale.

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Disclaimer

- The slides presented here are obtained / extracted from the authors of the prescribed books and from various other subject contributors.
- I hereby thank and acknowledge all the contributors for their material and inputs.
- I have added and modified a few slides to suit the requirements of the course.

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Module 8: Encoding and Feature Creation

Module 8 Topics

Topic 1: Need for encoding

Topic 2: One-hot encoding

Topic 3: Label encoding

Topic 4: Derived Features

Topic 5: Date-time feature extraction

Ref: T1 , Ch 6

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 1: Need for Encoding

Why is encoding needed?

- **Encoding** is essential because machines don't understand raw human language representation of data, categories, or labels. They only understand numbers.
- Encoding is the bridge that converts messy, human-friendly data into structured, machine-friendly form that are suitable for algorithms (whether simple statistics or advanced ML) to perform mathematical operations.
  - Words like "Red", "High", "Mumbai", "Yes" have no inherent numeric meaning.
  - Encoding converts them into numeric representations so that algorithms can compute distances, correlations, probabilities, etc.

Why encoding?

- Encoding Preserves Information
- Good encoding ensures that:
  - No information is lost
  - No artificial relationships are introduced
  - The model interprets categories correctly

- Example:
  - If you encode “Low, Medium, High” as 1, 2, 3 → the order is meaningful.
  - But if you encode “Red, Blue, Green” as 1, 2, 3 → the order is not meaningful.

- Choosing the right encoding preserves the semantics.

Why encoding?

- Encoding helps Models learn patterns
- Encoding transforms categories into mathematical structures that models can learn from.

- Examples:
  - One-hot encoding → helps models treat categories independently
  - Label encoding → useful for ordinal relationships
  - Target encoding → captures statistical relationships with the target
  - Embedding encoding → captures semantic similarity (e.g., in NLP)

Why encoding?

- Encoding Improves Model Performance
- The right encoding can:
  - Reduce noise
  - Improve accuracy
  - Speed up training
  - Reduce dimensionality
  - Avoid overfitting

- Encoding is not just preprocessing—it's a performance booster.

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 2: One-hot Encoding

One-hot encoding

- One-hot encoding is a technique used to convert categorical data into multiple flag column numeric format that machine-learning algorithms can understand.
- Instead of assigning numbers like 1, 2, 3 (which may wrongly imply order), one-hot encoding creates separate binary flag columns, one for each category value.
- Each row gets a 1 in the column that matches its category and 0 in all others.

Why do we use it?

- Prevents algorithms from assuming ordinal relationships where none exist
- Makes categories mathematically usable
- Creates grouped sets of categorical data
- Works well for models that rely on distance, linearity, or probability

Example

- Assume we have a column called Colour

Item No	Colour
101	Red
102	Blue
104	Red
105	Green
107	Blue
110	Red
112	Blue
113	Green
114	Blue
115	Red

Feature Colour has 3 distinct values

- After normal encoding, we get

Item No	Colour	Colour_Code
101	Red	1
102	Blue	2
104	Red	1
105	Green	3
107	Blue	2
110	Red	1
112	Blue	2
113	Green	3
114	Blue	2
115	Red	1

Assign a unique number to the Colour values

Example

- After One-hot encoding applied to Colour

Item No	Colour	Colour_Code	Colour_Red	Colour_Blue	Colour_Green
101	Red	1	1	0	0
102	Blue	2	0	1	0
104	Red	1	1	0	0
105	Green	3	0	0	1
107	Blue	2	0	1	0
110	Red	1	1	0	0
112	Blue	2	0	1	0
113	Green	3	0	0	1
114	Blue	2	0	1	0
115	Red	1	1	0	0

Observe, the dimension of the dataset is increased.

Each category becomes its own column.

Each row has exactly one “hot” (1) and the rest are “cold” (0).

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 3: Label Encoding

Label Encoding

- Label encoding is a technique that converts categorical values into numeric labels by assigning each unique category a number (usually starting from 0 or 1)
- Label encoding is useful when:
  - The categories have a natural order (ordinal data)
  - Examples:
    - Small < Medium < Large,
    - Beginner < Intermediate < Advanced
  - we want a compact numeric representation
  - The algorithm can handle integer-encoded categories (e.g., tree-based models)

Label Encoding

- When Should You Avoid Label Encoding?
- When categories do NOT have an order.
  - Example: Red, Blue, Green
  - If we encode them as 1, 2, 3 → the model may mistakenly assume Green > Blue > Red, which is meaningless.
- This is why one-hot encoding is preferred for nominal categories.

Example

Consider T-shirt sizes

Item No	Size
101	M
102	L
104	S
105	XL
107	M
110	L
112	L
113	S
114	XL
115	L

Item No	Size	Size_Code
101	M	2
102	L	3
104	S	1
105	XL	4
107	M	2
110	L	3
112	L	3
113	S	1
114	XL	4
115	L	3

Relationship / cardinality : XL > L > M > S

Example label encoding in Python

```
# Sample data
sizes = ["M", "L" , "S", “XL", "M", "L", “L", “S", “XL", “L"]

encoded_sizes = []

for s in sizes:
    if s == "S":
        encoded_sizes.append(1)
    elif s == "M":
        encoded_sizes.append(2)
    elif s == "L":
        encoded_sizes.append(3)
    elif s == "XL":
        encoded_sizes.append(4)
    else:
        encoded_sizes.append(None)   # for unexpected values

print("Original:", sizes)
print("Encoded :", encoded_sizes)
```

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 4: Derived Features

Derived Features

- Derived features (also called engineered features or constructed features) are new features created from existing data to improve analysis, to realize computed KPIs or to help a model learn better patterns
- They don't exist in the raw dataset - we derive them using logic, math, domain knowledge, or transformations.

Why we create derived features?

- Because raw data rarely captures the full story.
- Derived features help by:
  - Highlighting hidden patterns
  - Adding aspects specific too the KPIs of interest
  - Making relationships more explicit
  - Improving model accuracy
  - Reducing noise
  - Adding domain intelligence into the dataset

- In short: better features → better models.

Examples

1. Mathematical Transformations
- From Salary: create log(Salary) to reduce skew
- From Salary: create Commission
- From Age: create Age² to capture non-linear effects
- From Age: create YearsToRetire

2. Combining Columns
- From Height and Weight: create BMI
- From Price and Quantity: create Total_Spend

3. Date/Time Derivations
- From a timestamp: Calculate Day of week

4. Text-Based Features
From a review text:
- Calculate Word count
- Calculate Sentiment score
- Calculate Number of exclamation marks

Example creation of derived features in Python

```
import pandas as pd

# Sample dataset
data = {
    'Height_cm': [170, 160, 180],
    'Weight_kg': [70, 60, 90],
    'Price': [200, 150, 300],
    'Quantity': [2, 1, 4],
    'Date': ['2024-01-05', '2024-01-06', '2024-01-07']  # Fri, Sat, Sun
}
# create a dataframe from data
df = pd.DataFrame(data)

# Convert Date column to datetime
df['Date'] = pd.to_datetime(df['Date'])

# Derived Feature 1: BMI = weight / (height in meters)^2
df['BMI'] = df['Weight_kg'] / (df['Height_cm'] / 100) ** 2

# Derived Feature 2: Total Spend
df['Total_Spend'] = df['Price'] * df['Quantity']

# Derived Feature 3: Is Weekend
df['Is_Weekend'] = df['Date'].dt.dayofweek >= 5   # 5 = Saturday, 6 = Sunday

print(df)
```

innovate achieve lead

BITS Pilani

Pilani|Dubai|Goa|Hyderabad

Topic 5: Date-time feature extraction

Date-Time feature extractions

- Date and time columns contain lot of information than what appears at first glance.
- Date-Time Feature extraction helps to extract meaningful components that improve model performance, reveal patterns, and support better analysis.
- Especially significant when analyzing Time Series Data.
  - Maintaining ordinal relationship is important!
- Date-time details can be represented in many formats making it difficult for algorithms to understand (inspite of simple representation)
  - Ex: 05-06-2017 is to be interpreted as 5th June 2017 or 6th May 2017 ??

Why is it important?

- Because raw timestamps like: 2024-01-06 14:35:22 are not directly useful to a model.

- Hidden inside this single timestamp are dozens of potential features:
  - Day of week
  - Month
  - Quarter
  - Hour
  - Weekend indicator
  - Season
  - Time since previous event
  - And many more

- These derived features often capture behavioral patterns, seasonality, and cyclic trends.

Common Date Time Features to extract

1. Calendar-Based Features

These capture human and business cycles.

- Year
- Month
- Day
- Quarter (1–4)
- Week of year
- Day of week (0–6)
- Is weekend (True/False)
- Is month start / end
- Is quarter start / end
- Is year start / end

Useful for: sales forecasting, attendance prediction, seasonality analysis.

Common Date Time Features to extract

2. Time-Based Features

Extracted from the time portion.

- Hour
- Minute
- Second
- AM/PM
- Is business hour
- Is night time

Useful for: call center data, traffic data, energy consumption.

Common Date Time Features to extract

3. Duration / Time Difference Features

These capture intervals between events.

- Time since previous purchase
- Time until next event
- Days between signup and first purchase
- Delivery time = DeliveredDate – OrderDate
- Invoice age = today - InvoiceDate

Useful for: churn prediction, logistics, customer behavior.

Common Date Time Features to extract

4. Seasonal Features

Derived from domain knowledge.

- Season (Winter, Summer, Monsoon, etc.)
- Holiday indicator
- Festival indicator (Diwali, Christmas, Eid)
- School vacation period

Useful for: retail, tourism, transport

Common Date Time Features to extract

5. Rolling / Lag Features (for time series)

- Lag 1, Lag 7, Lag 30 (if data is daily, Lag 30 = value 30 days ago)
- Rolling mean (also called moving average)
- Rolling sum
- Rolling standard deviation

Useful for: forecasting, anomaly detection.

Date-Time Extractions Python code

```
import pandas as pd

# Sample data with timestamps
data = {
    'Timestamp': [
        '2024-01-05 14:30:00',
        '2024-01-06 09:15:00',
        '2024-01-07 20:45:00'     ]
}

df = pd.DataFrame(data)

# Convert to datetime
df['Timestamp'] = pd.to_datetime(df['Timestamp'])

# --- Date Extractions ---
df['Year'] = df['Timestamp'].dt.year
df['Month'] = df['Timestamp'].dt.month
df['Day'] = df['Timestamp'].dt.day
df['DayOfWeek'] = df['Timestamp'].dt.dayofweek      # 0 = Monday
df['WeekOfYear'] = df['Timestamp'].dt.isocalendar().week
df['Quarter'] = df['Timestamp'].dt.quarter
df['IsWeekend'] = df['Timestamp'].dt.dayofweek >= 5

# --- Time Extractions ---
df['Hour'] = df['Timestamp'].dt.hour
df['Minute'] = df['Timestamp'].dt.minute
df['Second'] = df['Timestamp'].dt.second
print(df)
```

Rolling mean- python code

```
import pandas as pd

# Sample sales data
data = {'Sales': [10, 20, 30, 40, 50, 60]}
df = pd.DataFrame(data)

# Rolling mean with window size = 3
df['Rolling_Mean_3'] =
    df['Sales'].rolling(window=3).mean()

print(df)
```

Sales	Rolling_Mean_3
10	NaN
20	NaN
30	20.0
40	30.0
50	40.0
60	50.0

The first two rows are NaN because you need **3 values** to compute the first rolling average.

Others..

Feature Split

- Some columns may contain multiple pieces of information bundled together; requires splitting these columns into constituent parts
- Feature splitting means breaking one feature (column) into multiple smaller, more meaningful features so that a model can learn better patterns.
- It is a form of feature engineering where we decompose a complex or composite feature into simpler parts.
- It is like unpacking hidden information inside a single column.
  - Ex: Splitting FirstName, MiddleName and LastName from Name

Why do we split features?

- Because some columns contain multiple pieces of information bundled together. Splitting helps:
  - Improve model accuracy
  - Make patterns and data items clearer
  - Reduce noise
  - Convert messy data into structured form
  - Allow algorithms to learn relationships separately

Examples

1. Splitting Date/Time:

   From a single timestamp, 2024-01-06 14:35:22, we can split into Year, Month, Day, Hour etc

   This is one of the most common forms of feature splitting.

2. Splitting Full Names

   From: Name = "Richa Mahadev Sharma"

   Split into:

   First_Name = Richa

   Middle_Name= Mahadev

   Last_Name = Sharma

   Useful in customer analytics, identity matching, etc.

Examples

3. Splitting Addresses to constituent parts

   From: "12 MG Road, Bengaluru, Karnataka"

   Split into:

   - Street : MG Road
   - City: Bengaluru
   - State: Karnataka

   This is useful in location-based analysis

4. Splitting Product Codes

   From Electronic Product Code: "ELEC-MOB-2024"

   Split into:

   - Category = ELEC
   - SubCategory = MOB
   - ModelYear = 2024

   This is extremely useful in retail analytics.

Feature splitting example in Python

```
import pandas as pd

# Sample data
data = {
    'Full_Name': ['Richa Mahadev Sharma', 'Teja Ganesh Rao', 'Sid Rajan Menon']
}

df = pd.DataFrame(data)
#print before split
print('Before split')
print(df)
print()

# Split First_Name into 3 parts
df[['First_Name', 'Middle_Name' ,'Last_Name']] = df['Full_Name'].str.split(' ', n=2, expand=True)

#print after split
print('After split')
print(df)
```

Summary

- Discussed the need for Encoding in Data Preprocessing.
- Understood One-hot encoding mechanism
- Studied Label encoding techniques.
- Understood the need for derived features and studied different types of derived features.
- Understood Date-Time extraction techniques and their significance in Feature engineering.
- Discussed other feature engineering. techniques used in Data Preprocessing.
- Studied a few implementation demos.

Questions?